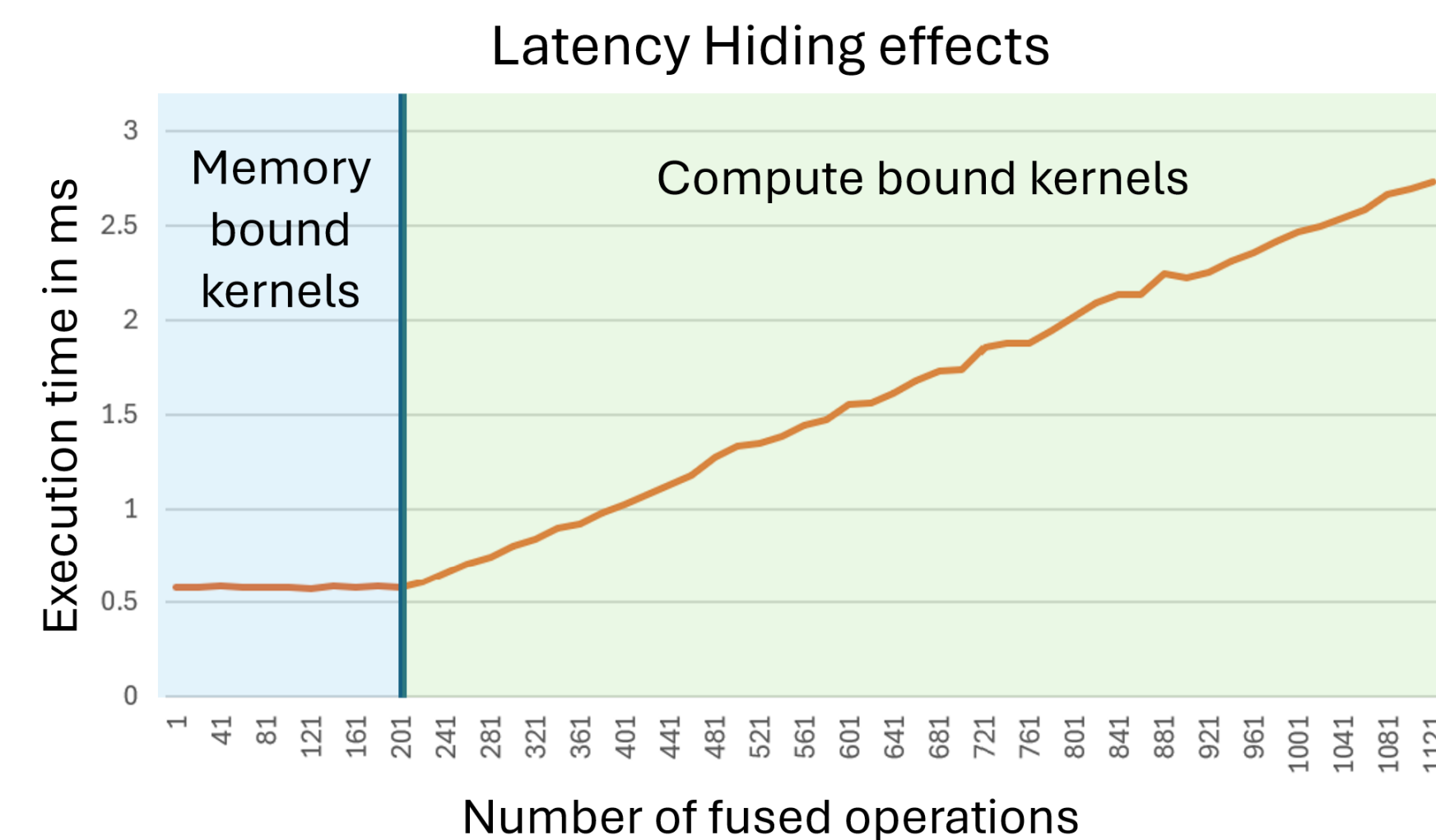
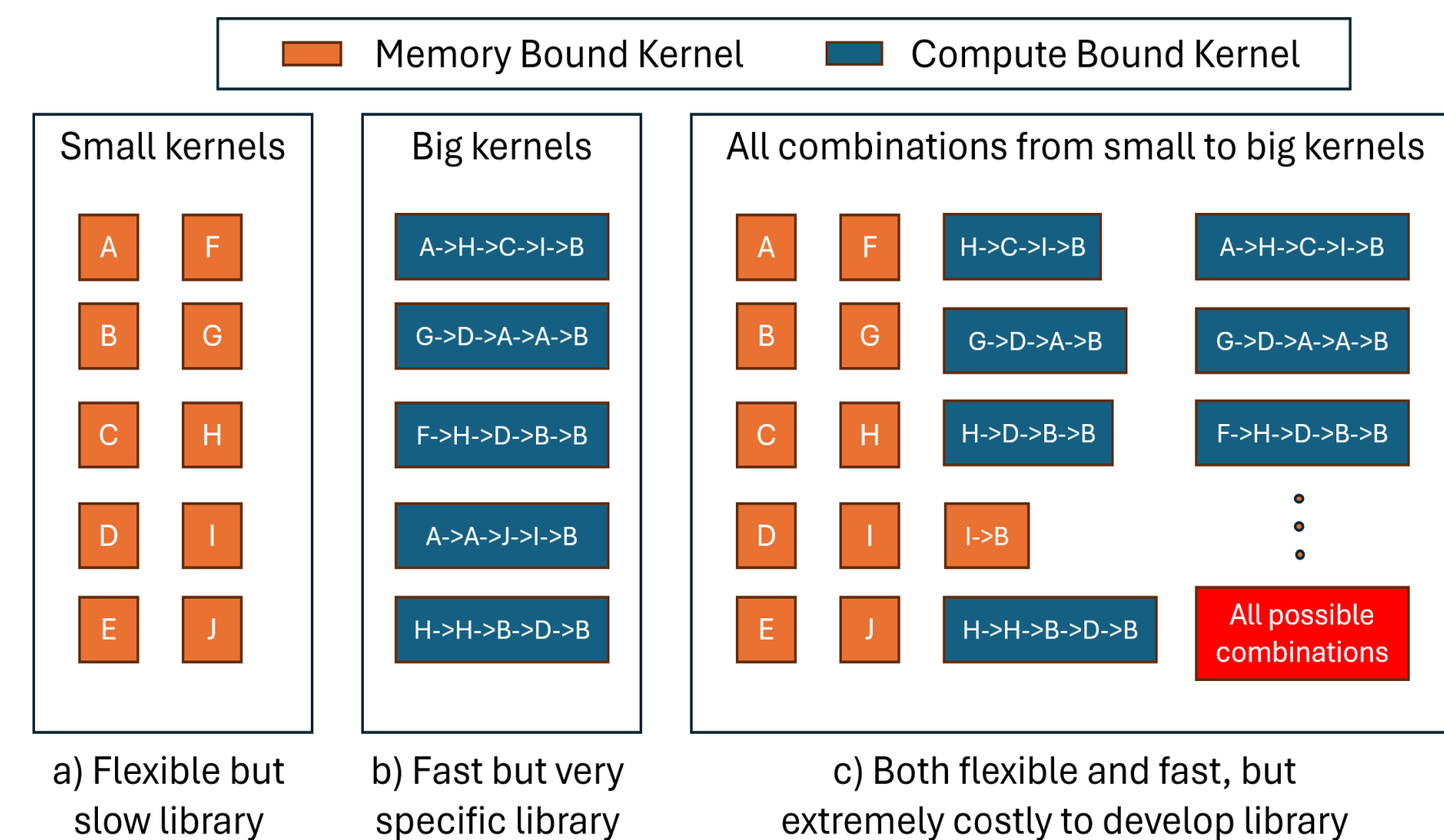


Problem



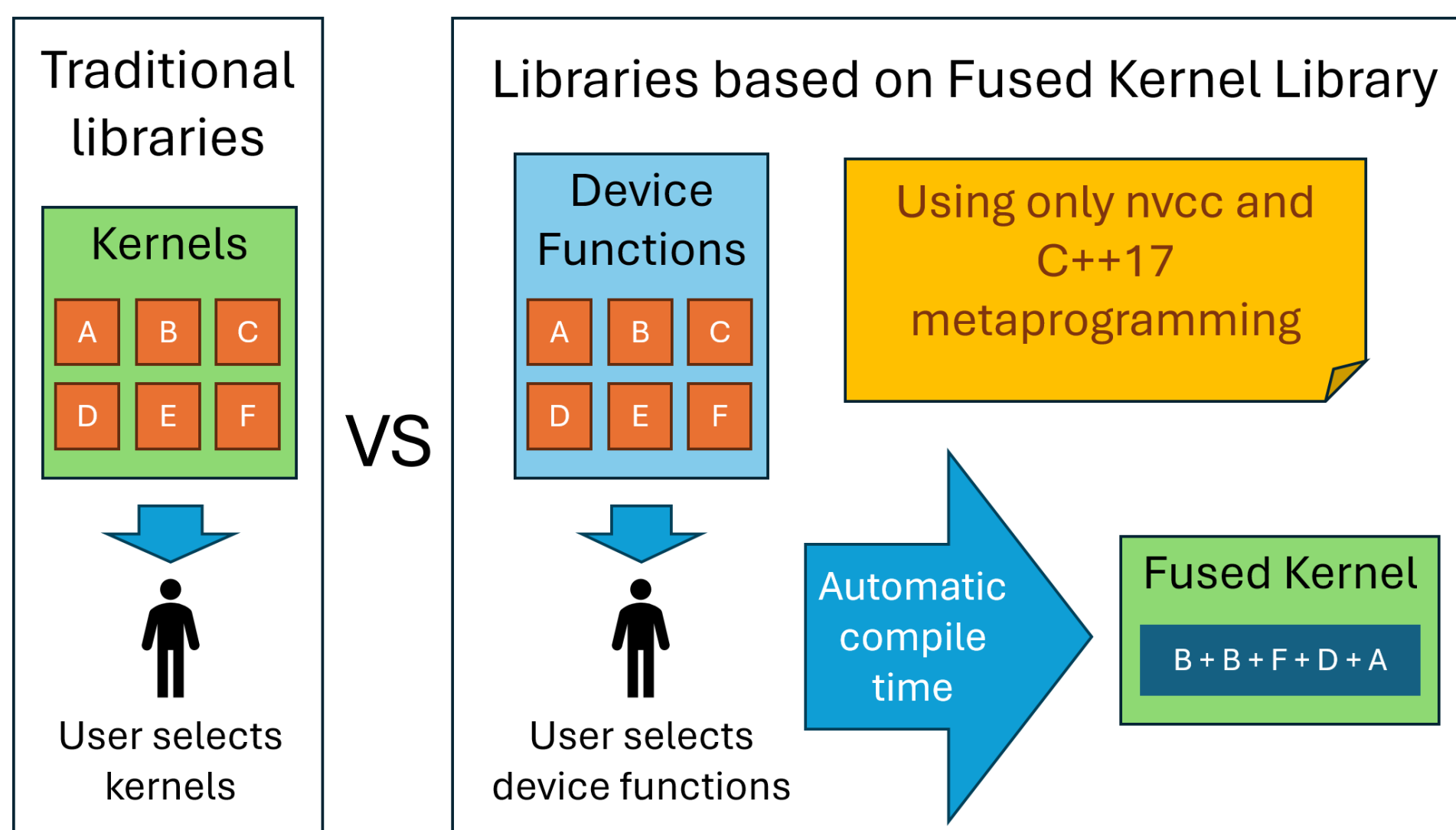
Executing a kernel that contains one operation, or 200 operations takes the same time (0,6ms).



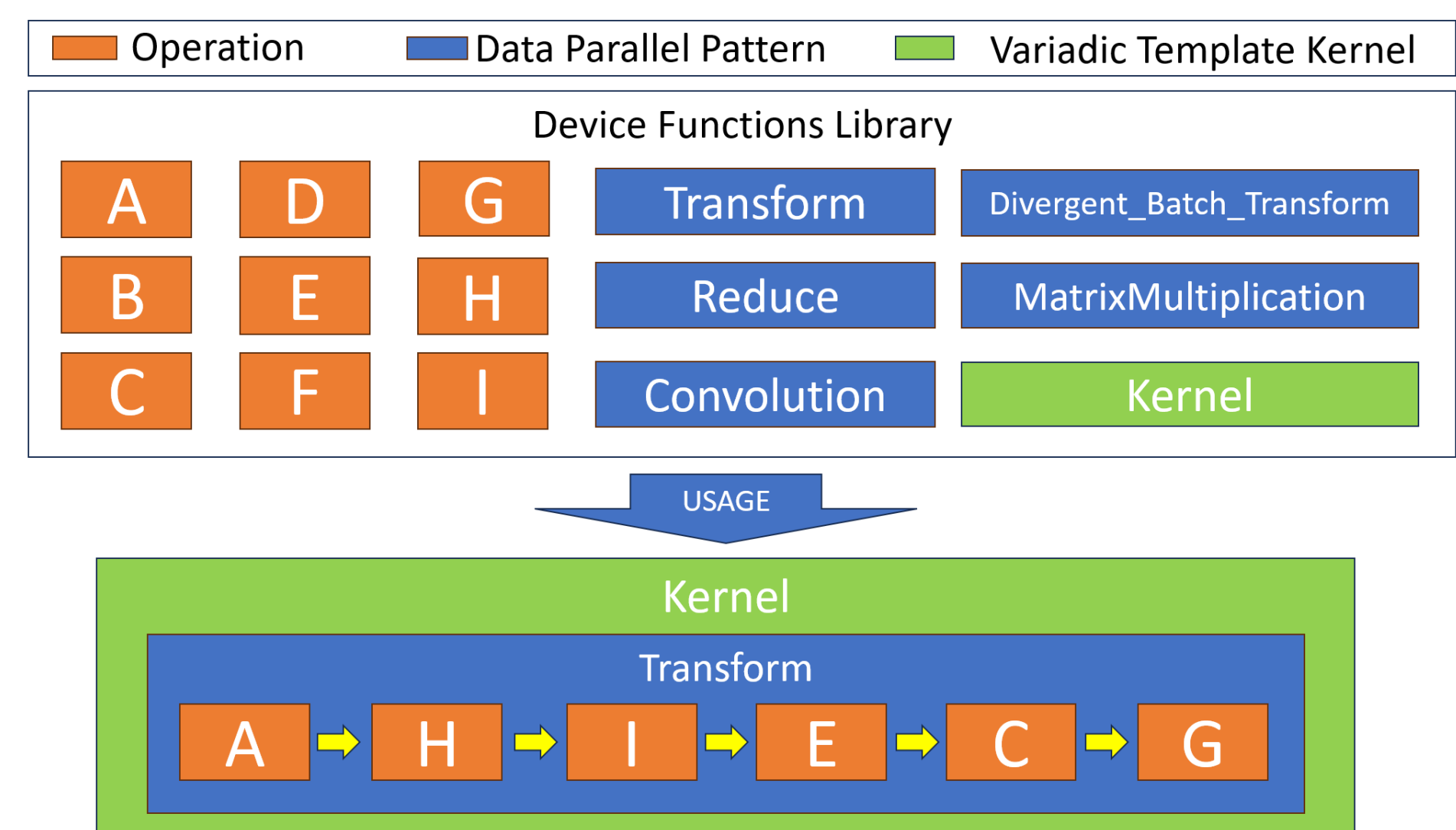
Many GPU libraries do not properly utilize the GPU's "latency hiding" feature, because they offer separated Memory Bound kernels (a). Alternatively they create hand made Vertically Fused Kernel combinations that may not suit all users (b). Doing all possible fused kernel combinations is impractical (c).

Our proposal

Let the user of the library define the Kernel Fusion, both Horizontal and Vertical. The compiler can generate the specified kernel, thanks to Static Polymorphism with templates.



Our solution creates a common code base for all libraries. It is possible to create several wrappers on top of it, to create the high level interfaces the final user is familiar with. Almost zero learning curve.



Results: seamless syntax, low learning curve

```
// OpenCV version
for (int crop_i = 0; crop_i < BATCH; crop_i++) {
    crops[crop_i].convertTo(crop_32f, CV_32FC3, 1, cv_stream);
    cv::cuda::resize(crop_32f, d_up, 0., 0., cv::INTER_LINEAR, cv_stream);
    cv::cuda::cvtColor(d_temp, d_temp, cv::COLOR_RGB2BGR, 0, cv_stream);
    cv::cuda::multiply(d_up, val_alpha, d_temp, 1.0, -1, cv_stream);
    cv::cuda::subtract(d_temp, val_sub, d_temp2, cv::noArray(), -1, cv_stream);
    cv::cuda::divide(d_temp2, val_div, d_temp, 1.0, -1, cv_stream);
    cv::cuda::split(d_temp, d_output_cv[crop_i], cv_stream);
}

// cvGPUSpeedup version (FKL based wrapper)
cvGS::executeOperations(cv_stream,
    cvGS::resize<CV_8UC3, cv::INTER_LINEAR, BATCH>(crops, up, BATCH),
    cvGS::cvtColor<cv::COLOR_RGB2BGR, CV_32FC3>(),
    cvGS::multiply<CV_32FC3>(val_alpha),
    cvGS::subtract<CV_32FC3>(val_sub),
    cvGS::divide<CV_32FC3>(val_div),
    cvGS::split<CV_32FC3>(d_output_cv)
);
```

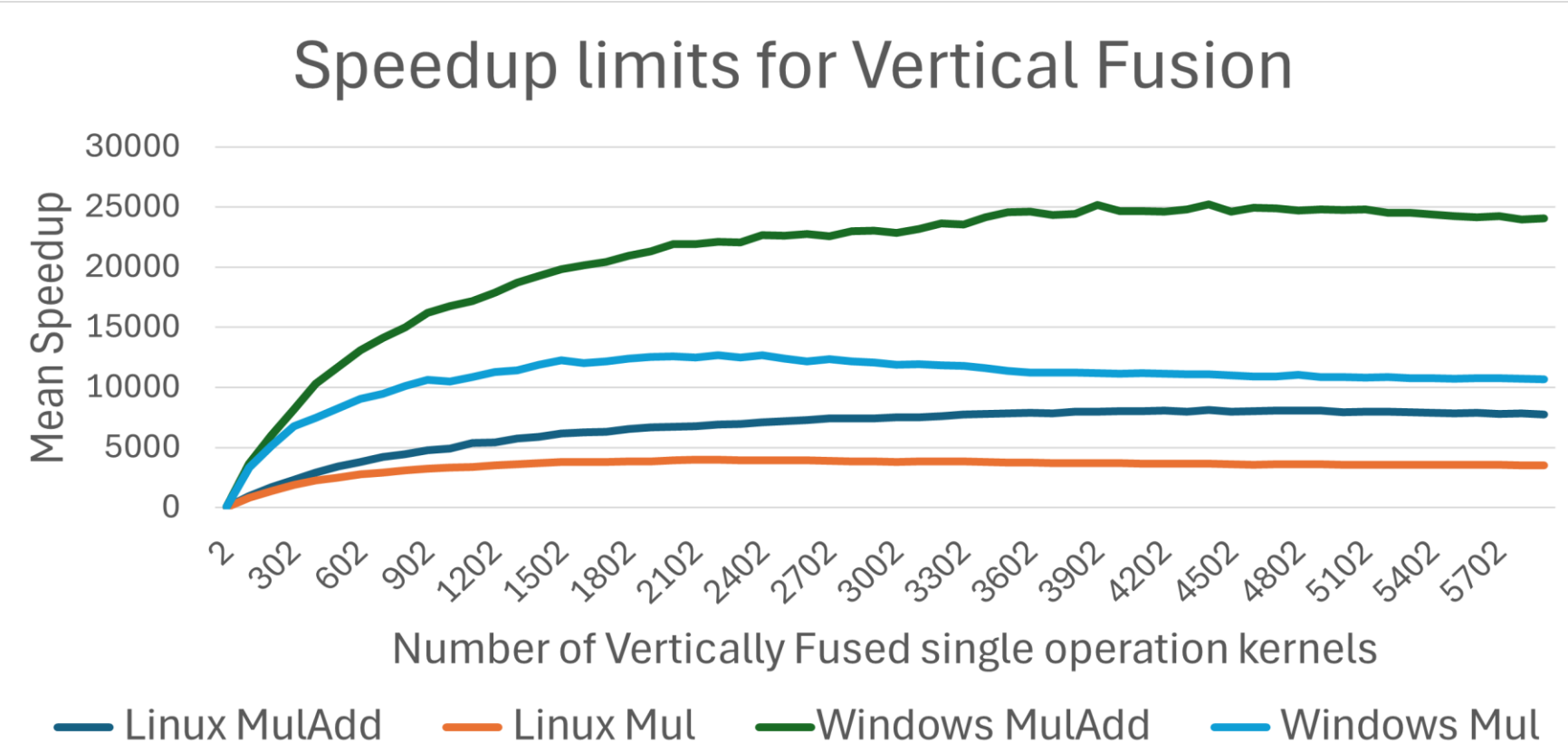
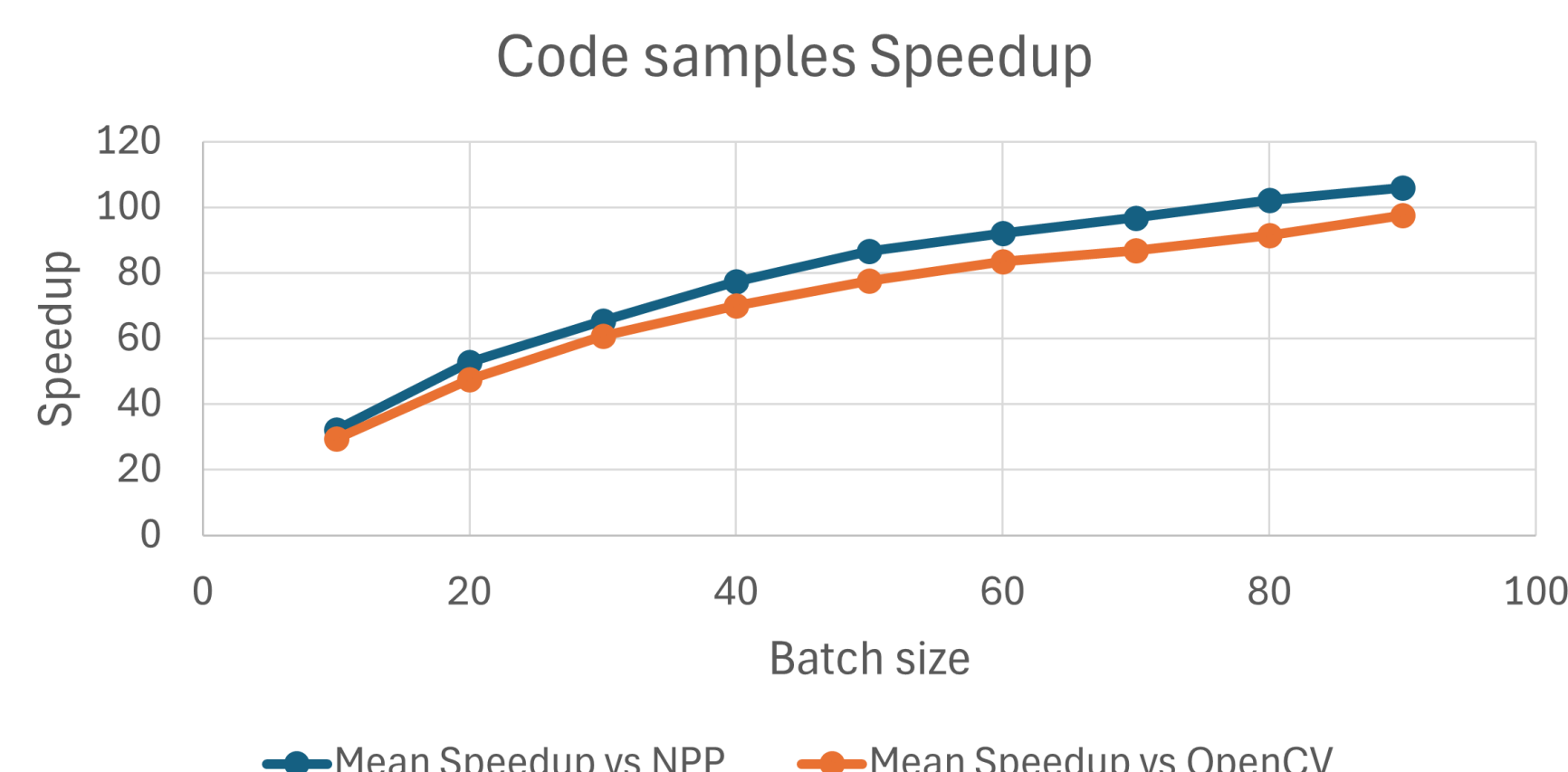
```
// NPP version
auto intMode = NppiInterpolationMode::NPPI_INTER_LINEAR
for (int i = 0; i < BATCH; ++i) { nppiConvert_8u32f_C3R_Ctx(hSrc[i], hSrcStep[i], hSrcFP32[i], hSrcFP32Step[i], crop_sz, nppCtx); };
nppiResizeBatch_32f_C3R_Advanced_Ctx(upW, upH, dSrc, dDst, ROI, BATCH, intMode, nppCtx);
for (int i = 0; i < BATCH; ++i) {
    nppiSwapChannels_32f_C3R_Ctx(hDst[i].pData, hDst[i].nStep, slmg.pData, slmg.nStep, up_sz, dstOrder, nppCtx);
    nppiMulC_32f_C3R_Ctx(slmg.pData, slmg.nStep, mulValue, mimg.pData, mimg.nStep, up_sz, nppCtx);
    nppiSubC_32f_C3R_Ctx(mimg.pData, mimg.nStep, subValue, slmg.pData, slmg.nStep, up_sz, nppCtx);
    nppiDivC_32f_C3R_Ctx(slmg.pData, slmg.nStep, divValue, dimg.pData, dimg.nStep, up_sz, nppCtx);
    nppiCopy_32f_C3P3R_Ctx(dimg.data, dimg.nStep, dstArr[i], dstStep[i], up_sz, nppCtx);
}

// FastNPP version (tentative FKL based wrapper)
fastNPP::executeOperations(nppCtx,
    fastNPP::ResizeBatch_8u32f_C3R_Advanced_Ctx<intMode, BATCH>(upW, upH, hSrc, ROI),
    fastNPP::SwapChannels_32f_C3R_Ctx(dstOrder),
    fastNPP::MulC_32f_C3R_Ctx(mulValue),
    fastNPP::SubC_32f_C3R_Ctx(subValue),
    fastNPP::DivC_32f_C3R_Ctx(divValue),
    fastNPP::Copy_32f_C3P3R_Ctx(dstArr, dstStep)
);
```

Variables not needed in cvGS/fastNPP

Variables that are the same

Results: big speedups



Results: real use case

OpenCV-CUDA

